

--	--	--	--	--	--	--	--	--

ESE: Nov-Dec 2012

Max. Marks: - 50

1. All questions are compulsory.
2. This is an open book test. You can read your textbooks/notes but can not discuss with anybody else.
3. Some questions may not have definite answers.
4. You can make assumptions if required, but you should state them.
5. Write code in C language. Code must be indented and commented.
6. You are allowed to explain your answers in Marathi or Hindi.
7. Notation for marks: [1*5] means 1 mark each for 5 questions.

- ```
int main() {
 int *p = (int *)
 malloc(sizeof(int) * 3);
 int *q = p + 1;
 q[0] = 10;
 p[2] = 20;
```

}

- Q.2 A)** Show the contents of the array after each iteration of selection sort [2\*5]  
for following data  
10 6 8 15 2
- B)** Derive equation for time taken by the following code:  

```
int main() {
 int i, j, n, k = 0;
 scanf("%d", &n);
 for(i = 0; i < n; i++)
 for(j = 0; j < n; j++)
 k++;

 return 0;
}
```
- C)** Write a function to find the value of second maximum of a set of numbers, stored in an array.  

```
int secondmax(int a[], int n);
// n is size of the array
```
- D)** Draw diagram of a hash table of size 7, after insertion of following data:  
data: 7 13 14 16 21 4  
Use the hashing scheme which uses linear probing and following hash function.  

```
int hash(int number) {
 return number % 7;
}
```
- E)** Write implementation of a queue, using array and two indices, with following init function and given type declarations. Only write the enqueue() and dequeue() functions.  

```
typedef struct queue {
 int arr[16];
 int f, r;
}queue;
void init(queue *q) {
 q->f = q->r = 0;
}
```
- Q.3 A)** Reconstruct a binary tree (draw the diagram), for which the inorder and preorder traversals are given below: [3\*5]  
inorder: b d a f e c  
preorder: a b d c e f
- B)** Write a recursive function to find the minimum value node in a tree. Use the following typedefs:  

```
typedef struct node
{
 int val;
 struct node *left, *right
} node;
typedef node *tree;
```
- C)** Write a complete program which reads strings stored in a file, and prints only those strings which have a given length. Both the file name, and length should be given as command line arguments. *You should use only open(), read(), write(), close() functions, use of fopen() and related functions is not allowed.*

- D)** Write a function to find the middle element of a singly linked NULL terminated list, by traversing the list only once. You are not allowed to change the list. For lists of even length, you can select any one as the middle element. Use the following declarations:

```
typedef struct node {
 int val;
 struct node *next;
} node;
typedef struct node *list;
node *findmiddle(list l);
```

- E)** Given following type definitions for a sparse matrix, and given that entries are stored in the array "arr" in *unsorted* order, write a function to add two sparse matrices and return the resultant sparse matrix. Other variables have same meaning as discussed in class.

```
typedef struct elem {
 int r, c, v;
}elem;
typedef struct spm {
 elem arr[128]; // Note: Entries are not in sorted order
 int nrow, ncol, nelem;
}spm;
spm *spmadd(spm *a, spm *b);
```

- Q.4 A)** Implement a hashing scheme, such that each bucket has three slots, and chaining is done on a hash collision. Use the given data type definitions. The "name" field is the key. You must write a constant time hash function. **[5\*4]**

```
typedef struct entry{
 char name[16];
 int rollno;
}entry;
typedef struct bucket {
 entry slots[3];
 int nentries; //no.of used slots
 struct bucket *next;
}bucket;
bucket ht[13];
```

- B)** Write a generic function, using function pointers, to reverse an array of elements of any type. You should declare proper function pointer types. Write at least two examples of the functions, which can be passed as function pointers in your code.
- C)** Write a recursive function to print all numbers with a given count of 'numbers'(digits, bits, etc) and with a given 'base'.  
*For example, when called print(3,2) the function prints all numbers in base '2', and having a length of 3. That is*  
 000 001 010 011 100 101 110 111.  
*Another example is print(2,4) which will print*  
 00 01 02 03 10 11 12 13.
- D)** Write a function to reverse a stack, by using only the push(), pop(), init(), empty(), full() and top() functions. *Note: This means, you have no idea about the stack implementation.*